PUC Minas

TRANSFORMAÇÃO E ARMAZENAMENTO DE DADOS

Prof. Victor Sales Silva
victorssbh@hotmail.com

Na aula de hoje...

- Limpeza, transformação e padronização de dados
- Integração, agregação e enriquecimento de dados
- Processamento com Python e PySpark
- Formatos, particionamento e organização de dados
- Armazenamento local e em cloud: S3, Azure Blob e GCS
- Prática

VAMOS COMEÇAR?

RESUMO

- Objetivos da aula
  - Transformar dados brutos em conjuntos adequados ao consumo analítico.
  - Diferenciar limpeza, transformação, padronização, integração e enriquecimento.
  - Implementar transformações utilizando Python e PySpark.
  - Realizar joins, agregações e derivações de atributos.
  - Escolher formatos adequados para armazenamento.
  - Compreender o impacto do particionamento no desempenho.
  - Organizar dados em estruturas locais e em Data Lakes.
  - Comparar S3, Azure Blob/ADLS Gen2 e GCS.
  - Projetar uma estrutura de armazenamento considerando **desempenho, custo, governança econsumo**.

Limpeza, transformação e padronização de dados

- A etapa de qualidade respondeu "Os dados estão confiáveis?



- Agora a pergunta é: "Como transformar esses dados para que sejam úteis ao negócio?".

Limpeza, transformação e padronização de dados

PROCESSO	OBJETIVO CENTRAL	EXEMPLO PRÁTICO
LIMPEZA	Remover inconsistências e valores nulos	Remoção de espaços em branco e trimming
PADRONIZAÇÃO	Uniformizar representações e formatos	Conversão de "Minas Gerais" para "MG"
TRANSFORMAÇÃO	Alterar estrutura, tipo ou valor	Derivar idade a partir do nascimento
INTEGRAÇÃO	Combinar múltiplas fontes de dados	Cruzamento entre Clientes e Pedidos
ENRIQUECIMENTO	Adicionar contexto externo ou cadastral	Enriquecimento de CEP com Município/UF
AGREGAÇÃO	Modificar granularidade para resumos	Cálculo do total de vendas por mês

Limpeza, transformação e padronização de dados

BI & DASHBOARDS

Agregações pré-calculadas, tabelas Fato/Dimensão e esquemas Star Schema.

MACHINE LEARNING

Encoding categórico, normalização de escala e engenharia de features.

AUDITORIA & FINANÇAS

Preservação do histórico imutável, rastreabilidade e reconciliação.

Limpeza, transformação e padronização de dados

Categorias técnicas de transformação

1. ESTRUTURAIS

Renomeação, seleção de colunas, reordenação e operações de Pivot / Unpivot.

2. SEMÂNTICAS

Categorização, tradução de códigos de domínio e interpretação de status.

3. NUMÉRICAS

Escalonamento, arredondamento e derivação de valores (quantidade × preço).

4. TEMPORAIS

Extração de ano/mês/dia, cálculo de durações e janelas operacionais.

5. RELACIONAIS

Joins entre tabelas, lookups cadastrais e enriquecimento relacional.

Estruturar transformações em camadas reutilizáveis simplifica a manutenção.

Limpeza, transformação e padronização de dados

Padronização e Normalização

Tratamento de Strings e Mapeamento Condicional

MAPEAMENTO CONDICIONAL EM PYSPARK

```
from pyspark.sql.functions import when, col, trim, upper
df = df.withColumn( "uf_padronizada", when(col("uf").isin("SP", "sp", "São Paulo"), "SP") .when(col("uf").isin("MG", "mg", "Minas Gerais"), "MG") .otherwise(upper(trim(col("uf")))) )
```

OPERAÇÕES FREQUENTES

trim(), upper(), lower(), regexp_replace().

FONTES DESPADRONIZADAS

Planilhas manuais, cadastros legados e APIs externas sem contrato rígido.

Limpeza, transformação e padronização de dados

Padronização e Normalização

Harmonização de Unidades e Decomposição Temporal

CONVERSÃO DE UNIDADES

Conversão para uma escala base unificada antes da soma ou média.

2 kg + 2000 g → 2 kg + 2 kg = 4 kg

DECOMPOSIÇÃO TEMPORAL

Extração de dimensões para análise de sazonalidade e agrupamento.

"2026-10-06" → Ano: 2026, Mês: 10, Trimestre: 4

Integração, agregação e enriquecimento de dados

Integração multi-fonte e desafios estruturais

ESTRUTURA DIFERENTE

Mapeamento de chaves divergentes (ex: cliente_id vs id_cliente).

SEMÂNTICA DIVERGENTE

Códigos com significados distintos (ex: 1 = Ativo vs A = Ativo).

CONFLITO DE GRANULARIDADE

Misturar dados agregados por pedido com itens individuais sem normalizar.

Integração, agregação e enriquecimento de dados

Tipos de JOIN e Controle de Cardinalidade

TIPO DE JOIN	COMPORTAMENTO
INNER	Apenas correspondências exatas
LEFT / RIGHT	Preserva integralmente um dos lados
LEFT ANTI	Identifica órfãos e ausência de chaves
LEFT SEMI	Filtra registros que possuem par

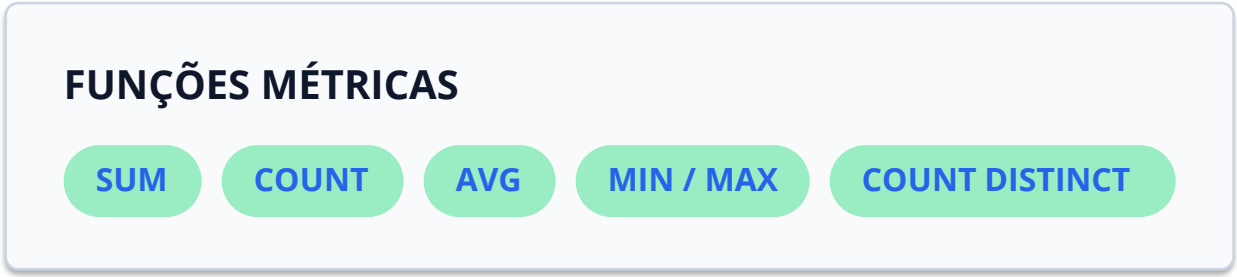
⚠ RISCO DE MULTIPLICAÇÃO DE LINHAS

Chaves duplicadas na tabela referenciada expandem a quantidade de registros involuntariamente (ex: 100 pedidos transformam-se em 135 linhas).

Integração, agregação e enriquecimento de dados

Funções métricas e definição de granularidade

- Granularidade é a precisão / nível de detalhe de **uma linha** na tabela final.
- Ex:
  - Item de Pedido → Pedido → Cliente → Mês.



Integração, agregação e enriquecimento de dados

Enriquecimento Interno, Externo e Feature Engineering

Dados Internos

Cruzamento entre ERP, CRM, E-commerce e registros de atendimento.

APIs Externas

Injeção de geolocalização, dados meteorológicos e indicadores econômicos.

Features de ML

Derivação de atributos comportamentais, recência e risco de fraude.

Processamento com Python e PySpark

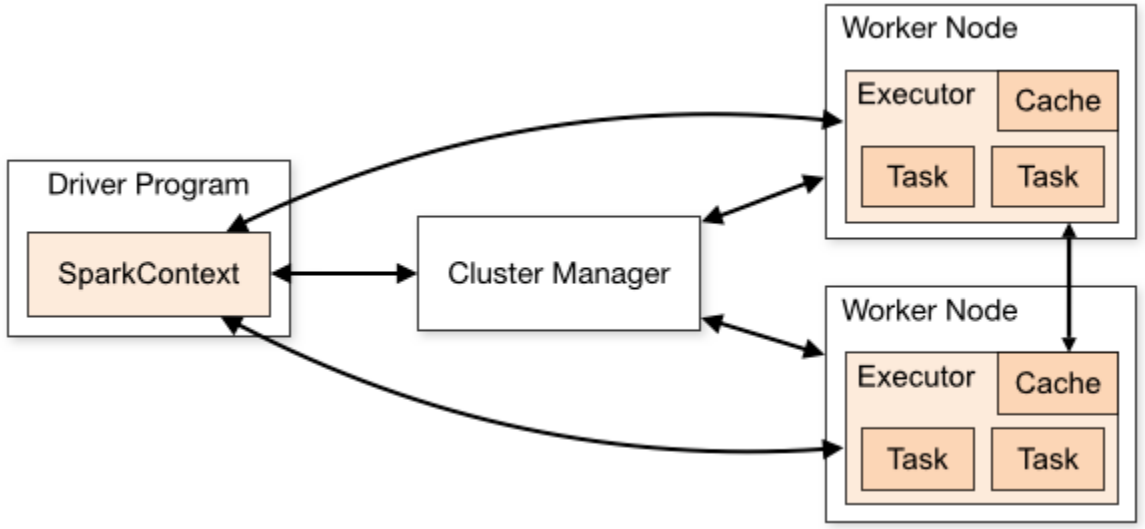
Arquitetura e Escala: Python vs Apache Spark

PYTHON / PANDAS

Execução em nó único limitada pela memória RAM. Ideal para prototipação e volumes pequenos.

APACHE SPARK / PYSPARK

Cluster distribuído com Driver/Executors, Lazy Evaluation e otimizador Catalyst para Petabytes.



Processamento com Python e PySpark

Pipeline prático de transformação em PySpark

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, upper, trim, sum
spark = SparkSession.builder.appName("PipelineAnalitico").getOrCreate()

# Transformação encadeada com Lazy Evaluation

resultado = spark.read.parquet("s3a://bronze/vendas/") \
            .filter(col("status") == "CONCLUIDO") \
              .withColumn("uf", upper(trim(col("uf")))) \
              .withColumn("valor_total", col("quantidade") * col("preco")) \
                  .groupBy("cliente_id", "uf") \
                      .agg(sum("valor_total") \
                          .alias("faturamento"))

resultado.write.mode("overwrite").parquet("s3a://gold/faturamento_cliente/")
```

Formatos, particionamento e organização de dados

Comparativo entre formatos

ATRIBUTO	CSV / JSON	APACHE PARQUET	DELTA LAKE
ORIENTAÇÃO	Linhas (Texto)	Colunar (Binário)	Colunar com Transaction Log
COMPRESSÃO & SCAN	Baixa / Scan Completo	Alta (Snappy) + Column Pruning	Alta + Indexação Avançada
TRANSAÇÕES ACID	Não	Não	Sim (Garantia de consistência)
TIME TRAVEL	Não	Não	Sim (Histórico de versões)

Formatos, particionamento e organização de dados

Particionamento e resolução do Small Files Problem

PARTICIONAMENTO HIVE

Diretórios físicos no formato /ano=2026/mes=10/ evitam scans desnecessários.

SMALL FILES PROBLEM

Múltiplos arquivos menores que 128MB degradam o desempenho de I/O na nuvem.

COMPACTAÇÃO & Z-ORDER

Utilização de comandos OPTIMIZE e clustering multidimensional.

Armazenamento local e em cloud: S3, Azure Blob e GCS

Armazenamento Object Store

AMAZON S3

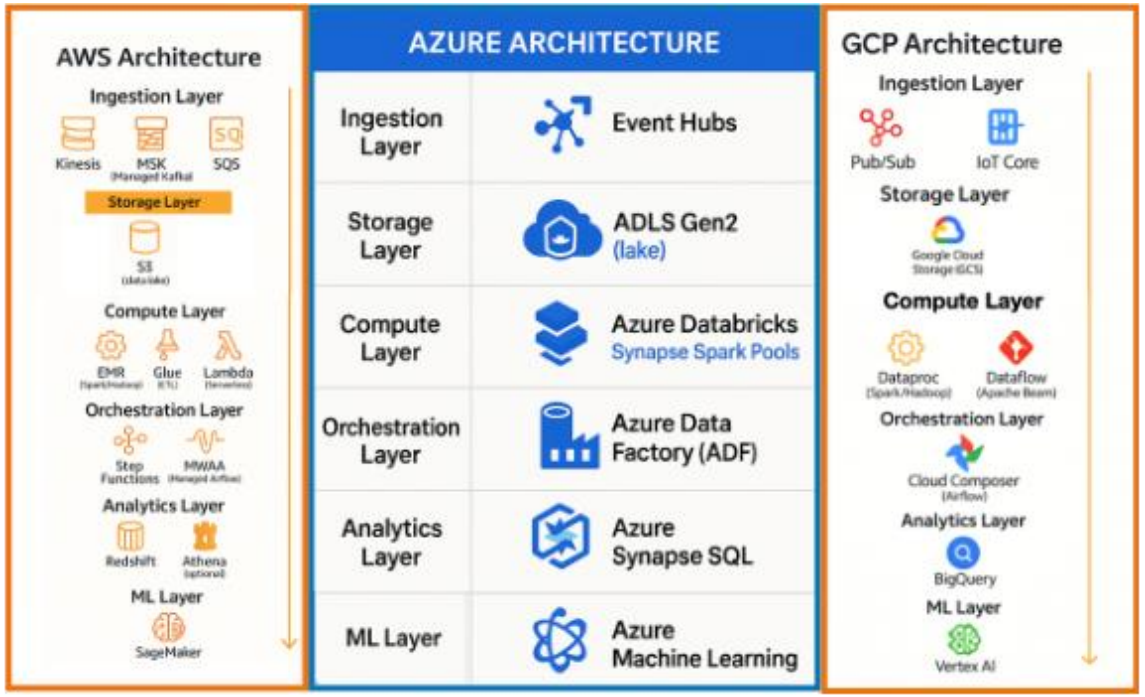
Padrão da indústria com buckets, prefixos e consistência forte.

AZURE DATA LAKE STORAGE (ADLS GEN2)

Namespace hierárquico otimizado para operações de diretório de analytics.

GOOGLE CLOUD STORAGE (GCS)

Integração nativa com ecossistema BigQuery e suporte a interconectividade.



Armazenamento local e em cloud: S3, Azure Blob e GCS

Organização física: Arquitetura Medalhão

Camada Bronze (Raw)

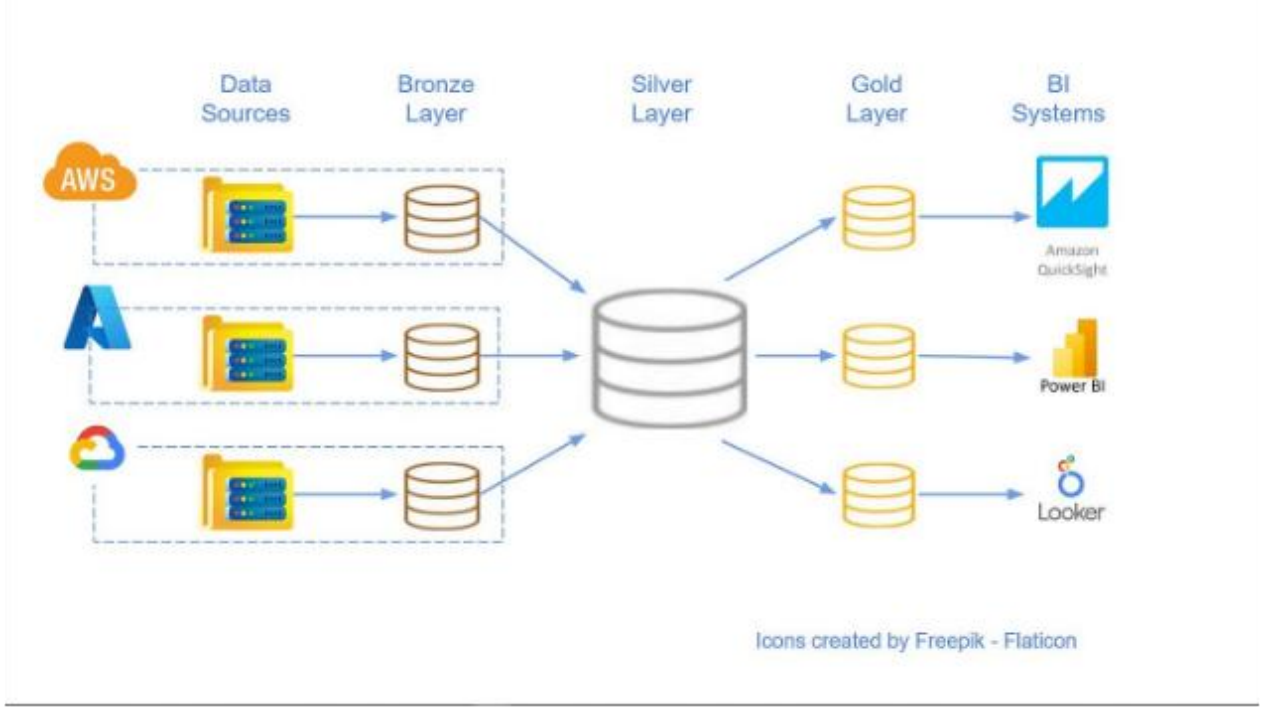
Dados brutos e imutáveis mantidos no formato original de ingestão.

Camada Silver (Cleaned)

Dados filtrados, limpos, desduplicados e padronizados.

Camada Gold (Curated)

Agregações de negócio prontas para consumo de BI e IA.



Armazenamento local e em cloud: S3, Azure Blob e GCS

Boas práticas para Analytics, IA e MLOps

DATA CONTRACTS

Garantia de schema e qualidade na interface de publicação.

DATA LINEAGE

Rastreabilidade completa da origem ao dashboard/modelo.

OBSERVABILIDADE

Monitoramento de volumetria, schema drift e anomalias.

DÚVIDAS?

OBRIGADO!

Referências bibliográficas

- CRICKARD, Paul. **Data Engineering with Python**: work with massive datasets to design data models and automate data pipelines using Python. 1. ed. Birmingham: Packt Publishing, 2020.

- CHAMBERS, Bill; ZAHARIA, Matei. **Spark: The Definitive Guide**: Big Data Processing Made Simple. 1. ed. Sebastopol: O'Reilly Media, 2018.

- DELTA LAKE. **Delta Lake Documentation**. São Francisco: Linux Foundation, 2026. Disponível em: https://docs.delta.io/. Acesso em: 06 out. 2026.

- REIS, Joe; HOUSLEY, Matt. **Fundamentals of Data Engineering**: Plan and Build Robust Data Systems. 1. ed. Sebastopol: O'Reilly Media, 2022.